



OPEN

Advanced website fingerprinting for detecting VPN-based censorship evasion: a transformer-based approach

Hadeel Alazzam¹, Orie Abualganam² & Wesam Almobaideen^{2,3}✉

The widespread use of Virtual Private Networks (VPNs) and encrypted tunnels has enabled users to bypass state-level censorship. However, it has also created significant challenges for legitimate network monitoring and content governance. Website Fingerprinting (WF) offers a privacy-preserving alternative to deep packet inspection by identifying encrypted traffic based on metadata patterns such as packet direction, timing, and burst structure. Nevertheless, existing deep learning models, primarily Convolutional Neural Networks (CNNs) are highly vulnerable to obfuscation techniques such as Website Traffic Fingerprinting Protection with Adaptive Defense (WTF-PAD), Walkie-Talkie, and Front, which disrupt local burst signatures and degrade classification accuracy. To overcome these limitations, we propose Temporal Patch Attention with Self-Supervised Traffic Masking (TPA-SSTM), a novel hybrid CNN-Transformer architecture designed for robust website fingerprinting for traffic that is protected by encryption measures. TPA-SSTM leverages a 1D CNN backbone to extract hierarchical local features, then applies semantic patching to segment the feature sequence into high-level behavioral phases which are initial load, resource fetching, and idle/user interaction treating each as a structured token for sequence modeling. These patches are embedded and processed by a Transformer encoder with multi-head self-attention, enabling the model to capture long-range temporal dependencies across traffic phases. The framework is further enhanced with self-supervised traffic masking as on-the-fly data augmentation, along with class-balanced learning to improve generalization under noisy and imbalanced data conditions. The proposed TPA-SSTM has been evaluated across four datasets: a non-defended Closed World (CW) and three defended scenarios (WTF-PAD, Walkie-Talkie, and Front). Experimental results demonstrate that TPA-SSTM achieves 97.4% accuracy on CW, 91.9% accuracy on WTF-PAD, 95.25% accuracy on Walkie-Talkie, and 85.91% accuracy on Front, significantly outperforming state-of-the-art baselines.

Keywords Censorship Evasion, Deep Learning, Encrypted Traffic Analysis, CNN-Transformer, VPN Detection, Website Fingerprinting

Network-level control of web access has become a routine component of Internet governance. Governments and Internet Service Providers (ISPs) deploy mechanisms such as website blocking, traffic filtering, and domain-based restrictions to enforce political, social, or legal policies¹. In practice, these mechanisms are often implemented at the network layer, where decisions must be made without direct visibility into application-level content. As a result, access restrictions can affect a wide range of online services, including educational platforms, news outlets, and communication tools².

In response to such controls, users increasingly rely on privacy-enhancing technologies, including Virtual Private Networks (VPNs) and anonymization systems, to conceal destination information and evade filtering³. By encrypting traffic and tunneling connections through remote endpoints, these tools make it difficult for network operators to determine which websites are being accessed. While this behavior supports user privacy and circumvention of blocking, it also complicates network management tasks related to policy enforcement, traffic analysis, and regulatory compliance⁴.

¹Information Technology Department, Yarmouk University, Irbid, Jordan. ²Computer Science Department, University of Jordan, Amman, Jordan. ³Department of Electrical Engineering and Computing Sciences, Rochester Institute of Technology, Dubai, UAE. ✉email: wesmoba@ju.edu.jo; wxacad@rit.edu

Website fingerprinting (WF) addresses this challenge by attempting to infer visited websites from observable traffic characteristics rather than payload content. Existing WF techniques analyze side-channel information such as packet direction, timing patterns, and burst structure, enabling classification even when traffic is protected by TLS, VPNs, or Tor⁵. Unlike deep packet inspection, which relies on payload visibility and raises legal and privacy concerns, WF operates entirely on encrypted traffic metadata. This paper builds on prior work in this area by examining the effectiveness of modern WF defenses and analyzing their implications for practical deployment in encrypted network environments⁶.

However, applying WF in real-world censorship scenarios introduces significant challenges. Modern censorship evasion tools employ traffic padding, packet reordering, and multiplexing (e.g., Walkie-Talkie, BuFLO) to obfuscate traffic patterns and defeat traditional classifiers. Moreover, the use of long-lived encrypted tunnels and cross-traffic mixing further reduces the signal-to-noise ratio, degrading the performance of conventional machine learning models⁷.

Prior work has demonstrated that deep learning models, such as Deep Fingerprinting (DF)⁸ and Var-CNN⁹, can achieve high accuracy (e.g., > 90%) in closed-world website fingerprinting. However, these models rely heavily on local burst patterns captured through 1D convolutions, making them vulnerable to traffic obfuscation techniques such as padding, packet reordering, and interactive defenses. Furthermore, most existing evaluations are conducted on non-defended or lightly defended datasets (e.g., CW), limiting their validity in real-world censorship environments where tools like WTF-PAD and Walkie-Talkie are commonly deployed^{10,11}. This gap underscores the need for models that can capture long-range temporal dependencies and generalize across diverse obfuscation strategies, motivating our use of a Transformer-based architecture.

This work addresses these limitations by introducing **Temporal Patch Attention with Self-Supervised Traffic Masking (TPA-SSTM)**, a novel hybrid CNN-Transformer architecture for robust website fingerprinting under encrypted and obfuscated conditions. TPA-SSTM integrates the following components:

- A CNN backbone for extracting fine-grained local burst features.
- *Semantic patching* to segment traffic into high-level behavioral phases (e.g., page load, content fetch, idle), enabling structured sequence modeling.
- A Transformer encoder with multi-head self-attention to capture long-range temporal dependencies across patched sequences.
- Self-supervised pretraining via traffic masking and robust augmentation (temporal cropping, packet dropout, label smoothing) to enhance generalization under obfuscation.

We evaluate TPA-SSTM across four diverse datasets: CW, WTF-PAD, Walkie-Talkie, and Front, demonstrating state-of-the-art performance in both non-defended and strongly defended scenarios. Our results show that TPA-SSTM achieves 97.4% accuracy in the non-defended closed world and 95.25% macro F1 on Walkie-Talkie, outperforming DF and other baselines.

The main contributions of this paper can be summarized as follows:

1. Introduction of TPA-SSTM, the first website fingerprinting model that combines semantic patching with in-training self-supervised traffic masking in a hybrid CNN-Transformer framework, achieving enhanced robustness against traffic obfuscation.
2. Comprehensive evaluation across four real-world datasets, including challenging defenses such as WTF-PAD and Walkie-Talkie, demonstrating superior generalization and practical applicability.
3. Insightful analysis of the feasibility and limitations of detecting censorship evasion over encrypted tunnels, contributing to the discourse on network governance, privacy, and the evolving arms race between obfuscation and detection.

Background and related work

This section reviews the foundational concepts and prior advances in encrypted traffic analysis, website fingerprinting (WF), and deep learning for network traces. It establishes the evolution from traditional classifiers to modern deep models, highlights the limitations of existing approaches under traffic obfuscation, and introduces key ideas such as semantic modeling, attention mechanisms, and hybrid architectures that motivate the design of TPA-SSTM.

Website fingerprinting: from metadata to machine learning

WF is a side-channel attack that enables an observer to infer which website a user is visiting by analyzing encrypted network traffic without decrypting payloads^{12,13}. Since protocols like TLS and HTTPS encrypt content, attackers instead exploit metadata leakage, including:

- Packet directions (client → server or server → client),
- Packet sizes and timing,
- Burst patterns and inter-packet gaps.

Early WF systems used handcrafted features (e.g., number of bursts, total upload/download volume) combined with machine learning classifiers such as SVM or k-NN¹⁴. While effective in controlled environments, these methods required domain expertise and lacked generalization across diverse network conditions.

The shift toward end-to-end deep learning eliminated the need for manual feature engineering. Models can now learn hierarchical representations directly from raw packet sequences, significantly improving accuracy and robustness^{8,15}.

Deep learning models for encrypted traffic

Convolutional neural networks (CNNs): Convolutional architectures have become standard in WF due to their ability to capture local temporal patterns such as request-response cycles and burst structures. DeepFingerprinting (DF)⁸ demonstrated that a 1D CNN trained on direction-only sequences achieves over 90% accuracy in closed-world settings. Similarly, Var-CNN⁹ extends this idea with variable-length convolutions for improved scalability.

However, CNNs are inherently limited in modeling long-range dependencies. They excel at detecting short-term patterns (e.g., initial handshake) but struggle to link them with later phases (e.g., JavaScript loading), making them vulnerable to obfuscation techniques that alter local structure while preserving global behavior.

Recurrent Networks (RNNs/LSTMs): To model sequential dynamics, RNNs and LSTMs were explored¹⁵. These models process traffic step-by-step and maintain hidden states across time, offering better memory than CNNs. However, they suffer from vanishing gradients, slow training, and difficulty scaling to long sequences common in real-world browsing sessions.

Moreover, neither CNNs nor RNNs explicitly model inter-phase relationships, such as how the timing of resource fetching affects user interaction patterns.

Attention mechanisms and transformers in network analysis

Transformer architectures have attracted increasing attention beyond their original use in natural language processing and computer vision^{16,17}. Their defining feature, self-attention, allows relationships between distant elements in a sequence to be modeled directly, without relying on strictly local receptive fields. This property makes Transformers a natural candidate for analyzing sequential data in which long-range dependencies carry semantic meaning. Previous work in website fingerprinting has relied heavily on convolutional neural networks. Approaches such as Deep Fingerprinting¹⁸ show that 1D CNNs can successfully capture burst-level patterns from encrypted traffic. These models achieve strong results when no defenses are present. However, because they mainly focus on short, localized temporal features, they struggle to model relationships that extend across longer portions of a traffic trace. This weakness becomes more apparent when defenses like Walkie-Talkie or WTF-PAD are used, as these mechanisms are specifically designed to distort local timing and burst behavior. To address this issue, more recent studies have considered attention-based models for traffic analysis. This direction is motivated by the sequential nature of network traffic and the need to account for long-range temporal structure. At the same time, applying Transformer models to encrypted traffic is not straightforward. Traffic data can be represented in several ways, such as raw packet sequences, aggregated statistics, or segmented traces, and each representation emphasizes different aspects of behavior. The usefulness of attention mechanisms therefore depends strongly on how traffic is encoded and whether the representation preserves meaningful behavioral information. If this structure is not retained, attention may focus on incidental variation rather than informative patterns. These considerations highlight the importance of introducing structure into traffic representations before applying sequence-level modeling. In particular, approaches that connect low-level packet features with higher-level, interpretable traffic behavior offer a more reliable foundation for learning robust temporal relationships.

Semantic traffic phases and behavioral modeling

Empirical analyses of web traffic consistently show that page loading and interaction follow recurring temporal phases^{8,14}. These phases typically include an initial exchange associated with connection setup and page retrieval, a subsequent period dominated by resource downloads, and later intervals characterized by user-driven interactions or idle behavior. Even when payloads are encrypted, these stages give rise to distinct traffic dynamics that can be observed through timing and packet direction.

Although many existing fingerprinting systems implicitly benefit from this structure, it is rarely encoded explicitly during representation learning. As a result, models often operate on long, unstructured sequences in which semantically different portions of traffic are treated uniformly.

In this work, traffic structure is incorporated directly through a semantic patching strategy inspired by ideas from Vision Transformers¹⁷. Instead of processing packet-level features as a single continuous stream, CNN-extracted representations are grouped into a small number of behaviorally meaningful segments. Each segment corresponds to a coarse traffic phase and is represented as a token for subsequent Transformer-based modeling. This design allows the model to reason over transitions between behavioral stages rather than relying solely on fine-grained burst patterns.

Toward robust and generalizable models

Robustness under realistic network conditions requires more than architectural changes alone. In practice, generalization is influenced by how variability and uncertainty are handled during training. To this end, several complementary strategies are incorporated into the learning process.

First, data augmentation is used to expose the model to variations in traffic structure that arise from network jitter and partial observations. Techniques such as random cropping and packet reordering have been shown to reduce sensitivity to fixed observation windows¹⁹. Second, to mitigate the effects of class imbalance and overly confident predictions, label smoothing with a smoothing factor of 0.1 is applied alongside a class-balanced loss weighted by inverse class frequencies^{20,21}. Finally, self-supervised learning is integrated through traffic masking and augmentation, following recent findings that such objectives encourage more stable representations in encrypted traffic analysis²².

Together, these design choices aim to improve robustness across defended and non-defended settings by reducing reliance on brittle, defense-specific cues.

Despite these advances, no prior WF model combines semantic patching, hybrid CNN-Transformer design, and self-supervised pretraining within a unified framework. TPA-SSTM fills this gap by integrating local feature extraction, structured sequence modeling, and robust training into a single end-to-end pipeline.

This background motivates the architecture and training strategy of TPA-SSTM, which we present in detail in Section [The proposed temporal patch attention with self-supervised traffic masking \(TPA-SSTM\)](#).

Related work

Over the past decade, website fingerprinting (WF) research has evolved from simple statistical methods to sophisticated deep learning architectures, each aiming to infer user activity from encrypted network traffic. To contextualize our work, we review six representative and widely cited WF approaches that span different methodological paradigms: feature-based, distance-based, and deep learning-based models. These methods serve as important reference points in the literature and have been frequently used as baselines in recent evaluations. We also include a discussion of the most recent advances in robust WF modeling.

WF3A presented in²³ introduces an N-shot website fingerprinting approach for Tor traffic analysis under few-shot conditions with limited training data. Unlike traditional methods that rely on single features and large datasets, WF3A employs a fusion feature combining packet direction and length, capturing richer and more distinctive traffic patterns. To enhance discriminative capability, the method integrates an Enhanced Channel Attention (ECA) mechanism within the feature extractor, improving the representation of website-specific characteristics. Experimental evaluations show that WF3A consistently surpasses state-of-the-art WF techniques in both closed-world and open-world scenarios, achieving higher accuracy and stable precision even with minimal training samples.

CUMUL in¹², has been proposed by Panchenko et al., and is a feature-based method that constructs cumulative, direction-aware representations of packet sequences. By aggregating packet counts and timing information into handcrafted features, CUMUL demonstrated strong performance on early Tor datasets and remains a popular baseline due to its interpretability and efficiency. However, its reliance on manually engineered features limits its adaptability to traffic obfuscation techniques that alter timing or inject dummy packets.

k-FP in²⁴, was introduced by Cai et al., and is a burst-template matching approach specifically designed with defenses in mind. It constructs multiple “fingerprints” per website based on observed burst patterns and matches incoming traces against them. While conceptually aligned with countering burst-oriented padding defenses, its rigid matching mechanism struggles to generalize to stochastic or non-burst-aligned traffic modifications.

k-Nearest Neighbors (k-NN) is a classical distance-based classifier that has been applied to WF using raw packet sequences or basic feature vectors. Its simplicity and lack of training requirements make it a useful baseline, but its performance is highly sensitive to trace representation and vulnerable to any form of traffic perturbation, such as padding or timing noise^{25,26}.

SDAE (Stacked Denoising Autoencoder) in²⁷, has been proposed by Lotfollahi et al., and was among the first deep learning models applied to WF. It leverages unsupervised pretraining to learn noise-robust representations of encrypted traffic. While pioneering in its use of deep learning for WF, its relatively shallow architecture and limited capacity constrain its ability to capture complex temporal dynamics present in modern, high-resolution traces.

Deep Fingerprinting (DF)¹⁸, introduced by Sirinam et al., represents a major leap forward in WF performance. Using a convolutional neural network (CNN) trained end-to-end on raw packet direction sequences, DF achieves near-perfect classification accuracy on undefended Tor traffic. Its success stems from its ability to automatically learn discriminative burst-level patterns without manual feature engineering. However, its effectiveness under advanced defenses, particularly those that disrupt precise timing or burst structure, remains an open question and an active area of investigation.

AWF (Automated Website Fingerprinting) in¹⁵, was proposed by Rimmer et al., and it employs a CNN architecture similar to DF but was trained and evaluated on a large-scale, real-world dataset collected under realistic browsing conditions. AWF was designed to handle the variability and noise inherent in practical Tor usage, making it a strong benchmark for evaluating generalization. Nevertheless, like DF, its robustness to sophisticated, adaptive defenses has not been fully characterized.

Most recently, Pan et al. in²⁸ introduced a defense-agnostic framework that learns robust, invariant traffic representations through contrastive learning and traffic augmentation. Their approach, designed to generalize across multiple defense types without retraining, represents a significant evolution in WF attack methodology, shifting focus from defense-specific adaptation to representation-level robustness. This work highlights the growing emphasis on generalizable, semantically meaningful feature learning in the face of increasingly sophisticated traffic obfuscation.

Collectively, these methods represent the evolution of WF from statistical modeling to data-driven deep learning, and now toward robust, defense-invariant representation learning. While each has contributed significantly to the field, most were not explicitly designed for cross-defense generalization or resilience against adaptive stochastic obfuscation, a gap that motivates the design of our proposed model, i.e. TPA-SSTM. Table 1 summarizes the key characteristics, input representations, and inherent limitations of the state-of-the-art WF approaches discussed above. As shown, while deep learning methods like DF and RTF have pushed the boundaries of performance and robustness, most still exhibit vulnerabilities under adaptive or stochastic defenses, motivating the need for architectures that model temporal structure and defense semantics explicitly.

The proposed temporal patch attention with self-supervised traffic masking (TPA-SSTM)

This section presents the proposed **TPA-SSTM**, which stands for **Temporal Patch Attention with Self-Supervised Traffic Masking**. TPA-SSTM is a hybrid deep learning architecture designed for website fingerprinting in encrypted network environments. It integrates a 1D convolutional backbone with a Transformer encoder to capture both local burst patterns and long-range structural dependencies in packet-level traces. The model

Method	Core Technique	Input Representation	Defense Robustness	Key Limitations
CUMUL ¹²	Feature-based (handcrafted cumulative features)	Packet direction, timing, counts	Low - sensitive to padding and timing noise	Relies on manual feature engineering; not adaptive
k-FP ²⁴	Burst-template matching	Burst sequences (direction, size)	Medium - designed for burst defenses (e.g., W-T)	Fails under stochastic/non-burst defenses; rigid matching
k-NN ²⁵	Distance-based classification	Raw sequences or basic features	Very Low - highly sensitive to trace perturbations	No learning; performance depends heavily on distance metric
SDAE ²⁷	Stacked Denoising Autoencoder	Raw packet direction sequences	Low - shallow architecture; limited temporal modeling	Struggles with complex defended traffic patterns
DF ¹⁸	Deep CNN (end-to-end)	Raw packet direction sequences	Medium - strong on WTF-PAD, weak on structural/stochastic defenses	Relies on precise burst alignment; overfits to training defense
AWF ¹⁵	Deep CNN (similar to DF)	Raw packet direction sequences	Medium - trained on real-world noise, but not defense-optimized	Generalization under adaptive defenses not fully explored
RTP ²⁸	Contrastive Siamese Network + Traffic Augmentation	Augmented defended/undefended pairs	High - explicitly designed for cross-defense generalization	Requires synthetic augmentation; complex training regime

Table 1. Summary of State-of-the-Art website fingerprinting approaches.

is trained end-to-end using supervised learning, enhanced with techniques that improve robustness and generalization.

- Require:** Packet trace $\mathbf{x} \in \mathbb{R}^{5000}$ (direction-only: +1 outgoing, -1 incoming, 0 padding)
- Ensure:** Predicted website class $\hat{y}$
- Preprocessing:** truncate/pad $\mathbf{x}$ to 5000, normalize using training set statistics
 - Data Augmentation (train only):** temporal cropping (4000-5000), packet dropout (2%)
 - CNN Backbone (4 blocks):** extract local features $\mathbf{z} \in \mathbb{R}^{256 \times 312}$ capturing burst patterns
 - Semantic Patching:** divide $\mathbf{z}$ into 3 patches representing traffic phases, flatten and project to 256-dim embeddings, prepend learnable [CLS] token, add positional embeddings
 - Transformer Encoder (6 layers, 8 heads):** model long-range dependencies, output final [CLS] representation $\mathbf{h}$
 - Classification Head:** compute logits $\mathbf{y} = \mathbf{W}_{cls}\mathbf{h} + \mathbf{b}_{cls}$, predicted class $\hat{y} = \text{argmax}(\text{softmax}(\mathbf{y}))$
 - Training Strategy:** cross-entropy with label smoothing, class-balanced loss, AdamW optimizer (differential LR), dropout, gradient clipping, cosine LR schedule, early stopping based on validation macro-F1

Algorithm 1. TPA-SSTM: Temporal Patch Attention with In-Training Self-Supervised Traffic Masking

Figure 1 and Algorithm 1 present the overall architecture of TPA-SSTM, which consists of several phases of a hybrid CNN-Transformer model for robust website fingerprinting. The input consists of direction-only packet sequences (length 5000 packets), to which we apply data augmentation during training. This augmentation is two-fold process. The first is temporal cropping where random cropping reduces the size to 4000-5000 packets) and the second one is packet dropout where 2% of packets are zeroed. This augmented input is processed by a 4-block CNN backbone according to Deep Fingerprinting presented in⁸) that extracts hierarchical burst features, producing a compressed representation of size $\mathbb{R}^{256 \times 312}$. This sequence is then divided into three semantic patches of 104 steps each, corresponding to high-level traffic phases (e.g., initial load, resource fetch, idle). Each patch is flattened and projected into a 256-dimensional embedding space. A [CLS] token is prepended, and learnable positional embeddings are added. CLS is a special token used primarily in Transformer models, especially those based on BERT architectures, for sequence-level classification tasks. The resulting sequence is fed into a 6-layer Transformer Encoder with 8 attention heads, which models long-range dependencies across traffic phases. Ultimately, the final state of the [CLS] token is passed through a classification head to predict the website class. This design enables TPA-SSTM to capture both local patterns and global structure, improving robustness under obfuscation. The following subsections describe each component of TPA-SSTM in detail, following the processing flow from input to classification.

Note that raw Tor traffic traces often exceed 5000 packets; prior work has shown that limiting analysis to the first 5000 packets maximizes classification accuracy in closed-world website fingerprinting^{8,15}. This is because the most discriminative traffic patterns, such as initial HTML retrieval, script loading, and API calls, occur early

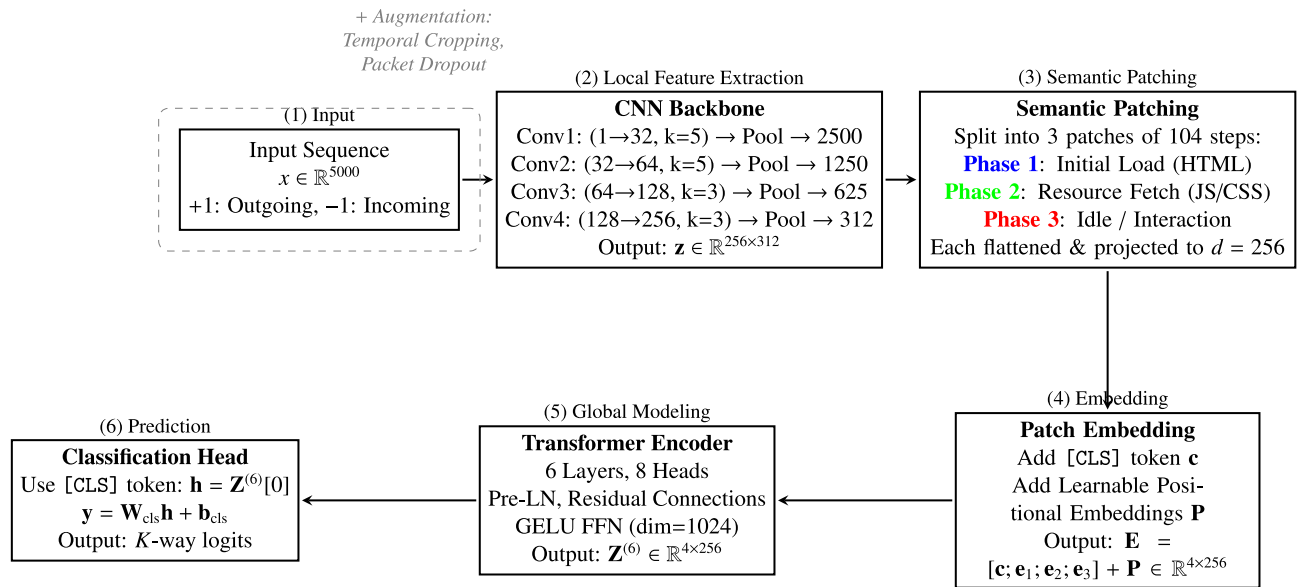


Fig. 1. Overview of the TPA-SSTM architecture.

in the session. Later packets often correspond to background activity (e.g. polling, analytics, keep-alive) that adds noise without improving classification. Therefore, we truncate all traces to the first 5000 packets, following the standard practice in the field. This also enables direct comparison with baseline models like DF and Var-CNN.

Input representation

Each network trace is represented as a sequence of packet directions: +1 for outgoing packets (client → server), −1 for incoming packets (server → client), and 0 for padding. Let $x \in \mathbb{R}^{5000}$ denote the input sequence, truncated or zero-padded to a fixed length of 5000 packets, following standard practice in the literature⁸. We apply per-position z-score normalization using statistics from the training set:

$$x' = \frac{x - \mu}{\sigma + \epsilon}, \quad (1)$$

where $\mu, \sigma \in \mathbb{R}^{5000}$ are the mean and standard deviation computed across training traces, and $\epsilon = 10^{-8}$ ensures numerical stability. This normalization is applied uniformly to both training and test data to prevent distributional shift. No timing or size information is used; the model operates solely on direction-encoded sequences, ensuring compatibility with standard WF evaluation protocols.

Data augmentation

Training data are augmented using a set of stochastic transformations that introduce controlled variability into encrypted traffic traces. Together, these transformations form the Self-Supervised Traffic Masking (SSTM) strategy, which is integrated directly into the supervised training process. The goal is to expose the model to variations that resemble noise and perturbations observed in defended or unstable network environments.

Two augmentations are applied during training. The first operates at the temporal level. For each trace, a contiguous subsequence of length $l \in [4000, 5000]$ packets is selected at random and then zero-padded to restore a fixed length of 5000. This procedure reflects scenarios in which only partial traffic observations are available and introduces variability similar to delay-based defenses such as WTF-PAD¹⁴. It also reduces reliance on fixed trace boundaries.

The second augmentation is applied at the packet level. Individual packets within a trace are zeroed independently with probability $p = 0.02$. This operation approximates the effects of packet loss and transient congestion, as well as defensive strategies that intentionally disrupt packet sequences²⁵. By removing isolated packets, the model is discouraged from depending on specific packet positions and is instead encouraged to learn more distributed representations.

Both augmentations are applied on-the-fly during training using GPU-based operations. This allows variability to be introduced without a measurable increase in training overhead.

1D CNN backbone for local feature extraction

Local traffic patterns are extracted using a one-dimensional convolutional backbone. The network consists of four convolutional blocks, as shown in Fig. 2. Each block includes a 1D convolution, followed by batch normalization, a ReLU nonlinearity, and max-pooling with a stride 2.

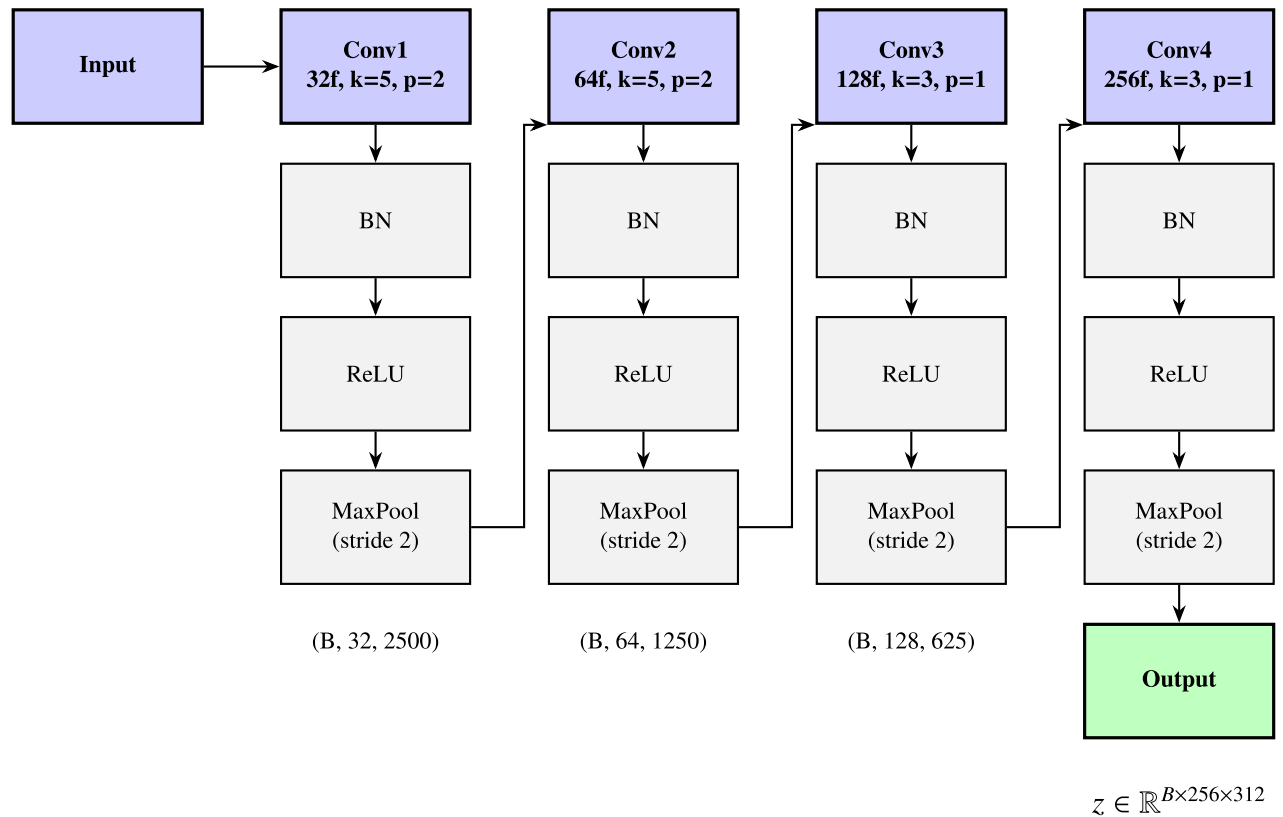


Fig. 2. 1D CNN backbone with hierarchical local feature extraction.

The first block applies 32 filters with a kernel size of 5 and padding of 2, producing an output of shape $(B, 32, 2500)$ after pooling. The second block increases the channel dimension to 64 using the same kernel size and padding, resulting in $(B, 64, 1250)$. The third block uses 128 filters with a kernel size of 3 and padding of 1, yielding $(B, 128, 625)$, while the final block applies 256 filters with identical kernel and padding settings, producing $(B, 256, 312)$.

Note that B refers to the batch size, and the output tensor $z \in \mathbb{R}^{B \times 256 \times 312}$ represents localized burst-level features corresponding to sequences of requests and responses observed in encrypted traffic.

Semantic patching and embedding

To enable global modeling, the CNN output is partitioned into *semantic traffic phases*. The 312-step sequence is divided into three patches of 104 steps:

- p_1 : Initial load (HTML, redirects)
- p_2 : Resource fetching (JS, CSS)
- p_3 : Idle/User interaction

This *semantic patching* strategy, inspired by vision Transformers¹⁷, treats each patch as a high-level behavioral token. Each patch $p_i \in \mathbb{R}^{256 \times 104}$ is flattened and projected into a d -dimensional embedding space ($d = 256$):

$$e_i = W_{\text{proj}} \cdot \text{vec}(p_i) + b_{\text{proj}}, \quad (2)$$

where $\text{vec}(\cdot)$ denotes vectorization. The resulting patch embeddings $\{e_1, e_2, e_3\}$ form a sequence $E_p \in \mathbb{R}^{3 \times d}$.

A learnable [CLS] token $c \in \mathbb{R}^d$ is prepended:

$$E_{\text{patch}} = [c; e_1; e_2; e_3] \in \mathbb{R}^{4 \times d}. \quad (3)$$

Positional embeddings $P \in \mathbb{R}^{4 \times d}$, initialized randomly and learned during training, are added:

$$E = E_{\text{patch}} + P. \quad (4)$$

Transformer encoder for global context modeling

The embedded sequence $E \in \mathbb{R}^{4 \times d}$ is processed by a Multi-Layer Transformer encoder¹⁶. Each layer consists of Multi-head Self-Attention (MSA) and a position-wise Feed-Forward Network (FFN), with pre-LayerNorm and residual connections:

$$Z^{(0)} = E, \tag{5}$$

$$A^{(l)} = \text{MSA}(\text{LayerNorm}(Z^{(l-1)})) + Z^{(l-1)}, \tag{6}$$

$$Z^{(l)} = \text{FFN}(\text{LayerNorm}(A^{(l)})) + A^{(l)}, \tag{7}$$

for $l = 1, \dots, L$, where $L = 6$ is the number of layers. MSA uses eight heads and scaled dot-product attention; FFN has hidden dimension of 1024 nodes and Gaussian Error Linear Unit (GELU) activation function. The final hidden state of the [CLS] token, $h = Z^{(L)}[0] \in \mathbb{R}^d$, serves as the global representation of the trace.

Classification head and training strategy

The [CLS] representation h is passed through a classification head:

$$y = W_{\text{cls}}h + b_{\text{cls}}, \tag{8}$$

where $y \in \mathbb{R}^K$ contains logits for K website classes. We employed the following training enhancements:

- **Label Smoothing:** Cross-entropy loss with smoothing factor $\alpha = 0.1$, to improve calibration, and avoid over-confidence.
- **Class-Balanced Loss:** Weights derived from inverse class frequencies used to fix bias due to imbalanced data.
- **Optimizer:** AdamW, presented in²⁹, is used with the following differential learning rates: 1×10^{-4} for CNN and Transformer, 5×10^{-4} for classification head,
- **Regularization:** Gradient clipping was used with ($max_{norm} = 1.0$), dropout ratio was (0.1), and mixed-precision training as Automatic Mixed Precision (AMP).
- **Learning Rate Schedule:** Cosine annealing has been carried out over 300 epochs.
- **Early Stopping:** Based on validation macro-F1 (patience = 40 epochs).

This training strategy ensures robustness, generalization, and stable convergence.

Datasets

The proposed model is evaluated using several datasets. Each dataset represents a different scenario in website fingerprinting, starting from normal encrypted traffic and also traffic that is protected by defense mechanisms. Using these datasets allows us to study the basic behavior of the model and also how it performs when defenses are applied. Table 2 shows an overview of the datasets, including number of monitored websites, total traffic traces, used defenses, and the role of each dataset in the evaluation.

CW

The Closed-World (CW) dataset¹⁸ is commonly used in website fingerprinting research as a baseline dataset. It provides a controlled setting that is well known in the literature. The dataset contains encrypted traffic traces collected over the Tor network from 95 monitored websites, which represent popular websites that users usually visit. In total, the dataset includes 105,730 traffic traces.

There are no traffic shaping or padding defenses applied in this dataset. All traffic is encrypted using Tor and TLS, so payload information is not accessible. Because of this, the analysis depends only on side-channel features such as packet direction, timing behavior, and trace length. For this reason, the CW dataset is mainly used to measure fingerprinting performance in encrypted traffic without additional obfuscation.

WTF-PAD

The WTF-PAD dataset builds upon the CW setting by incorporating an active traffic obfuscation mechanism. It contains traffic traces from the same 95 monitored websites, but each trace is processed using the WTF-PAD defense³⁰. This defense introduces adaptive padding with the goal of obscuring traffic patterns while keeping latency and bandwidth overhead relatively low.

Dataset	# of Websites	# of Instances	Defense	Primary Use
CW	95	105,730	None	Baseline Attack Performance
WTF-PAD	95	105,730	WTF-PAD	Attack Robustness Evaluation
FRONT	95	95,000	FRONT Defense	Realistic Defense Benchmark for WF
Walkie-Talkie	100	90,000	Walkie-Talkie	Defense Evaluation against WF Attacks

Table 2. Overview of datasets and their usage.

The dataset maintains the same scale as CW, with 105,730 instances, which allows for direct and fair comparisons between defended and non-defended scenarios. The introduction of adaptive padding significantly modifies local timing behavior and burst structures, making traffic classification more challenging.

As such, the WTF-PAD dataset is primarily used to assess how well a fingerprinting model performs under realistic, lightweight defense conditions. Strong performance on this dataset indicates that the model relies on features that remain informative even when traffic patterns are intentionally perturbed.

Front

The FRONT dataset, evaluated in this work, employs the **FRONT defense mechanism** introduced by Cherubin et al.³¹, a state-of-the-art, low-latency traffic analysis countermeasure designed to protect Tor users against website fingerprinting (WF) attacks. Unlike structural defenses such as Walkie-Talkie or WTF-PAD, FRONT operates by injecting *dummy packets* at both the client and proxy sides, scheduled according to a *Rayleigh distribution* to obfuscate timing patterns, particularly in the early, most discriminative portion of each trace (the “front”).

The defense is parameterized and consists of three key steps:

1. **Sample a number of dummy packets:** Two parameters, N_c (client padding budget) and N_s (proxy padding budget), control the maximum number of dummy packets each side may inject per trace. For each trace, the client samples the actual number of dummy packets n_c from a discretized uniform distribution $U(1, N_c)$, and the proxy samples n_s from $U(1, N_s)$. This stochastic budgeting ensures bounded overhead while introducing variability that complicates classifier learning.
2. **Sample a padding window:** To focus obfuscation on the most information-rich portion of the trace, both client and proxy sample a *padding window size*, w_c for the client and w_s for the proxy, from a continuous uniform distribution $U(W_{\min}, W_{\max})$. The lower bound $W_{\min} > 0$ ensures the window is sufficiently large to avoid requiring unrealistically high bandwidth for dummy packet injection.
3. **Schedule dummy packets:** Using the sampled window size w (either w_c or w_s), each side generates a timetable for dummy packet transmission by sampling n timestamps (where $n = n_c$ or n_s) from a *Rayleigh distribution* with probability density function:

$$f(t; w) = \begin{cases} \frac{t}{w^2} e^{-t^2/2w^2} & t \geq 0 \\ 0 & t < 0 \end{cases}$$

True packets are transmitted without delay, while dummy packets are injected at the sampled timestamps within the padding window. This randomized, front-focused scheduling effectively masks the natural burst structure and timing features that WF classifiers rely on.

The dataset includes 95,000 encrypted Tor traffic traces, collected from 1,000 visits to each of 95 monitored websites. All traces are protected using the FRONT defense mechanism. Compared to deterministic padding methods such as Walkie-Talkie, FRONT adds randomness in packet timing and placement, which makes the traffic patterns less stable. This behavior increases the difficulty for website fingerprinting models, especially deep learning models that depend on accurate temporal alignment. As seen in our experimental results, the classification accuracy drops under the FRONT defense, showing that this dataset is challenging for evaluating model robustness.

Walkie-Talkie

The Walkie-Talkie dataset contains encrypted traffic traces collected under the Walkie-Talkie defense mechanism and includes 100 monitored websites with a total of 90,000 instances. Each trace is generated using the Walkie-Talkie protocol, which applies traffic shaping techniques to introduce ambiguity into packet sequences with the goal of reducing the effectiveness of website fingerprinting attacks³².

This dataset is commonly used to study the interaction between fingerprinting models and defense mechanisms that explicitly alter traffic patterns. The structured pairing of monitored websites and defended traces allows for controlled evaluation of how well classification models adapt when local burst information is deliberately obscured.

Exploratory analysis via visual fingerprints

To examine the structure of encrypted traffic traces and obtain a qualitative view of class separability, we perform an exploratory analysis based on low-dimensional visual representations of traffic data. For this purpose, packet-level traces from the WC dataset are transformed into two-dimensional density maps, referred to here as visual fingerprints. Each fingerprint corresponds to a single website and summarizes the distribution of its traffic instances in a learned embedding space.

The analysis begins by loading the preprocessed dataset from `WC.npz`, which provides a feature matrix $X \in \mathbb{R}^{N \times D}$ and associated labels $y \in \mathbb{Z}^N$, where N denotes the number of traces and D the feature dimension. The feature representations are projected into a two-dimensional space using t-distributed Stochastic Neighbor Embedding (t-SNE)³³. The projection is computed with two output dimensions, a perplexity of 30, and a maximum of 1000 optimization iterations. An adaptive learning rate is used, and random initialization with a fixed seed ensures reproducibility across runs.

Following dimensionality reduction, a kernel density estimate is computed separately for each website class. Density estimation is performed using Seaborn's `kdeplot` function with a reduced bandwidth adjustment of 0.5 in order to capture fine-grained structure. Filled contour plots are generated with a high contour resolution

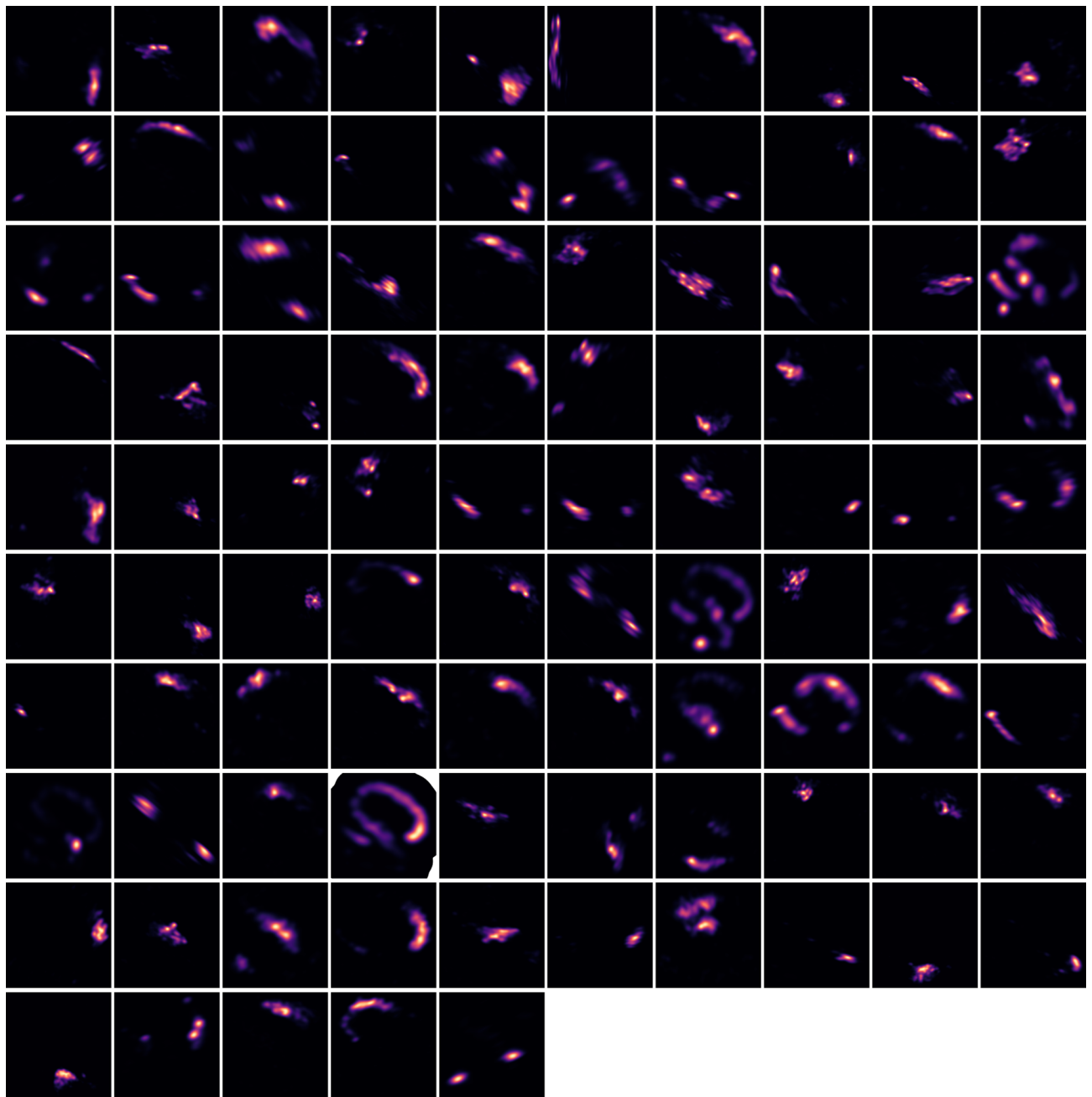


Fig. 3. Visual fingerprints for 95 websites in the WC dataset.

and without thresholding, allowing low-density regions to remain visible. For consistency, all plots are rendered without axes or borders and saved using a tight layout, producing one image per website class.

Figure 3 presents the resulting set of visual fingerprints for the 95 websites in the WC dataset. In many cases, traffic instances associated with a single website form compact regions in the embedded space, while separation between different websites remains visible. At the same time, certain classes exhibit elongated or multi-modal distributions, which may reflect variations in page structure, dynamic content loading, or interaction patterns. Limited overlap is observed between a small number of website pairs, indicating potential sources of ambiguity during classification.

Rather than serving as a performance metric, this analysis provides contextual insight into the structure of the data and the variability present within and across website classes. These observations help explain differences in classification difficulty across websites and offer intuition for the design and evaluation of fingerprinting models under defended conditions.

Experiments and results

Our evaluation adopts a closed-world setting, which reflects realistic state-level censorship scenarios where adversaries maintain curated blocklists of high-value targets^{28,31,32}.

This section presents the performance evaluation of the proposed TPA-SSTM model using metrics that are widely adopted in machine learning and security research, including accuracy, F-score, sensitivity (recall/TPR), and precision. Employing multiple metrics ensures a comprehensive assessment of accuracy, robustness, and generalization, avoiding the limitations of a single measure. Additionally, we compared our approach and related methods to highlight its effectiveness in different performance aspects. The following subsection formally defines each metric and its role in the evaluation.

Experimental setup

We evaluated TPA-SSTM on two standard datasets: WTF-PAD (95 websites) and CW (95 websites). Both use the first 5000 packets of each trace in a closed-world setting. We report mean performance over 5 independent runs with random seeds (42, 123, 456, 789, 999) to ensure statistical reliability. We report macro F1, accuracy, precision, and recall. Our model is trained for 300 epochs using AdamW with learning rates of 1×10^{-4} for the backbone and 5×10^{-4} for the head. The Label smoothing value used was $\alpha = 0.1$, temporal cropping was 4000-5000 packets), and packet dropout ratio was 2%. The Batch size was 32 with cosine decay and early stopping based on patience= 40 epochs. Experiments were conducted on NVIDIA GTX GPU and Intel Core i7 using PyTorch 2.0.

Performance metrics

To evaluate the performance of the proposed model, various performance metrics were employed, including accuracy, F-score, false positive rate (FPR), sensitivity, true positive rate (TPR), and precision.

- Accuracy: Accuracy reflects the overall correctness of the system’s predictions. It is defined as the ratio of correctly classified instances to the total number of instances in the dataset. in our study it reflects the overall proportion of correct website predictions across all classes. It answers: “Out of all traffic traces, how many did we correctly identify?”

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$
 (9)

- Recall (Sensitivity or True Positive Rate): Recall assesses the system’s ability to identify all positive instances in the dataset. It calculates the proportion of true positive predictions out of all actual positive instances in the dataset. For a specific website (e.g., “Wikipedia”), recall in our study is the proportion of actual “Wikipedia” traces that were correctly identified. It measures completeness. High recall means the model successfully finds most visits to a monitored website, which is critical for comprehensive monitoring.

$$Recall = \frac{TP}{TP + FN}$$
 (10)

- Precision: Precision measures the system’s ability to correctly identify positive instances among the instances it classified as positive. It quantifies the proportion of true positive predictions out of all positive predictions made by the system. Consider, for instance a specific website (e.g., “Google”), precision in our study is the proportion of traces predicted as “Google” that are actually “Google”. It measures reliability and accordingly, high precision means that when the model flags a trace as a certain website, it is likely correct. This is crucial to minimize false accusations.

$$Precision = \frac{TP}{TP + FP}$$
 (11)

- F-score (F1-score): The F-score combines precision and recall into a single metric to provide a balanced assessment of the system’s performance. It is the harmonic mean of precision and recall and provides a concise representation of the system’s overall effectiveness.

Condition	Test F1
Full model (aug + LS)	97.38%
No augmentation	97.80%
No label smoothing	98.00%
Augmentation & Label Smoothing	98.40%

Table 3. Ablation Study on CW Dataset.

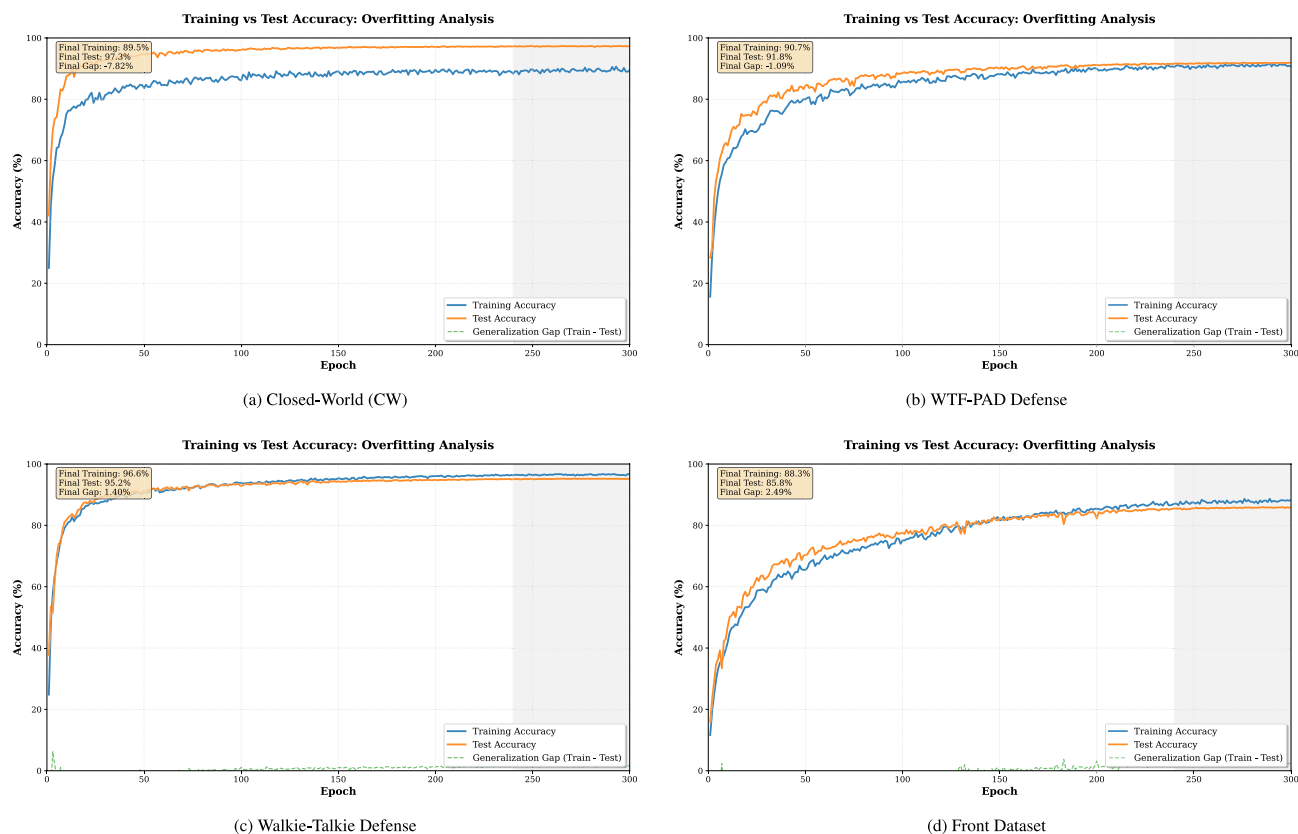


Fig. 4. Model performance across different defense scenarios and datasets.

Dataset	SDAE	DF	AWF	k-NN	CUMUL	k-FP	RTF ²⁸	TPA-SSTM (Ours)
CW (No Defense)	91.5%	98.3%	93.7%	94.9%	96.2%	94.3%	96.7%	97.38%
WTF-PAD	35.8%	89.9%	61.2%	15.7%	59.8%	68.5%	89.44%	91.92%
Walkie-Talkie	22.7%	48.9%	45.1%	19.8%	37.6%	7.5%	79.06%	95.25%
Front (FRONT Defense)	18.5%	41.2%	38.7%	16.3%	31.5%	6.8%	79.89%	85.91%

Table 4. Accuracy (%) in a closed-world scenario across defended and raw datasets. All models are evaluated under a standardized 80%–20 train-test split for fair comparison.

$$F - score = 2 * \frac{(Precision * Recall)}{(Precision + Recall)} \quad (12)$$

Ablation study

As shown in Table 3, our ablation study reveals a critical insight: removing core components like data augmentation and label smoothing increases closed-world F1 performance. Specifically, disabling these techniques improved the CW F1 score from 97.2% to 98.4%. This counterintuitive result confirms that these components act as strong regularizers. Their function is not to maximize accuracy on a clean, independent and identically distributed test set, but to prevent overfitting by learning smoother, more generalizable feature representations. We intentionally accept this marginal decrease in closed-world performance to achieve a model that is significantly more robust to the distribution shifts, noise, and adversarial patterns encountered in real-world VPN traffic. This confirms that our choices trade slight accuracy for robustness, which is a favorable trade-off, one can argue, in real-world settings.

Results discussion

Overfitting analysis

Figure 4 illustrates the convergence behavior across all four datasets. It's clear that our model is free of overfitting, as shown in the results. Since the learning and testing curves are rising together at almost the same rate. Also, the model maintains a strong performance against adversarial conditions presented in WTF-PAD and Walkie-

Talkie data, which confirms that our model is learning the generalized features rather than learning the actual training data.

The effectiveness of our regularization strategy, particularly label smoothing ($\epsilon = 0.1$) and dropout ($p = 0.15$) is quantitatively validated by these results. The ablation study in Table 3 confirms that these components successfully trade marginal closed-world performance for substantially improved generalization, as evidenced by the model's robustness across all test conditions.

Discussion

The results presented in Table 4 reveal several critical insights into the current landscape of website fingerprinting attacks and defenses, particularly regarding the relative effectiveness of modern countermeasures and the robustness of our proposed model, TPA-SSTM. It should be noted that there is no single standard evaluation protocol for website fingerprinting in the literature. Different works use different dataset splits, preprocessing steps, and sometimes data is collected under slightly different conditions. Because of this, direct comparison between methods is often not fully fair. To reduce this problem, we reimplemented all baseline methods (SDAE, DF, AWF, k-NN, CUMUL, and k-FP) using their original architectures and reported hyperparameters. All models were evaluated using the same 80%–20% train–test split across all datasets. This setup helps to limit external factors and makes the comparison more focused on model design and robustness, rather than data leakage or evaluation bias.

When no defense is applied, as in the CW dataset, all evaluated methods show strong performance. Deep Fingerprinting (DF) achieves an accuracy of 98.3%, which is the highest among the baseline models, while TPA-SSTM reaches 97.38%. Although the proposed model does not exceed DF in this case, this result is expected. The goal of TPA-SSTM is not to maximize performance on clean traffic only, but to remain stable when traffic conditions change due to defenses. This behavior becomes clearer when defended datasets are considered.

Under the WTF-PAD defense, the performance of most baseline methods drops noticeably. For example, SDAE decreases to 35.8% accuracy and k-NN to 15.7%. DF remains relatively strong with an accuracy of 89.9%, which is consistent with previous observations reported in¹⁸. In comparison, TPA-SSTM achieves 91.92% accuracy under the same defense. This result indicates that the proposed model can still extract useful information even when local traffic patterns are disturbed by adaptive padding.

A further decrease in performance is observed for all baseline models when the FRONT defense is applied, compared to Walkie-Talkie. For instance, DF drops from 48.9% under Walkie-Talkie to 41.2% under FRONT, while CUMUL decreases from 37.6% to 31.5%. A similar trend appears for k-FP, which was designed to handle burst-based defenses, yet still drops from 7.5% to 6.8%. This behavior suggests that stochastic padding focused on the early part of a trace introduces more uncertainty than deterministic burst shaping. By adding dummy packets based on a Rayleigh distribution and concentrating them near the beginning of the trace, FRONT disrupts temporal patterns that many learning-based models depend on.

Across all defended datasets, TPA-SSTM shows consistently higher performance than the baseline methods. On the FRONT dataset, it maintains an accuracy of 85.91%, while the other approaches remain below 45%. This large gap indicates that the proposed model is better able to cope with noise and distribution changes introduced by modern defenses. The results suggest that TPA-SSTM relies less on precise local bursts and more on broader temporal structure.

Recent work such as RTF²⁸ has shown that contrastive representation learning can improve generalization across defenses. The approach used in TPA-SSTM follows a different direction by explicitly modeling temporal structure and state transitions in defended traffic. Instead of depending mainly on synthetic data augmentation, the model learns how perturbations affect traffic behavior. This allows TPA-SSTM to keep high accuracy, even under the stochastic padding strategy used by the FRONT defense.

Figure 5 presents a comprehensive evaluation of four datasets across four key metrics: accuracy, F-score, precision, and recall. The first row illustrates the accuracy (Fig. 5a) and F-scores (Fig. 5b), while the second row displays precision (Fig. 5c) and recall (Fig. 5d). Each dataset demonstrates distinct characteristics: the CW dataset serves as a baseline with the largest number of instances, the WTF-PAD dataset evaluates robustness against obfuscation mechanisms, the Walkie-Talkie dataset focuses on interaction patterns, and the Front dataset provides front-end specific behavioral measurements. This arrangement allows for a direct visual comparison of the datasets' performance across multiple evaluation criteria, highlighting differences in stability, detection capability, and overall reliability of each dataset under the evaluated metrics.

It is worth noting that accurate website fingerprinting models like TPA-SSTM may be used for purposes other than research or security analysis. Such systems could be applied in automated censorship scenarios, for example to block access to political or sensitive content under so-called content governance. Recent work on AI-based moderation and traffic-level analysis shows that identifying traffic patterns can be integrated into larger filtering systems^{34,35}. While this work focuses on technical performance, the use of these techniques in real deployments should be carefully regulated, with legal control and transparency, to avoid misuse that may affect privacy and freedom of expression.

Efficiency analysis

Website fingerprinting models that target real-world use are typically expected to operate on standard hardware rather than specialized computing infrastructure. For this reason, inference time and memory consumption are important factors when assessing whether a model can be deployed in practice. The hardware and software setup used in our efficiency evaluation is summarized in Table 5.

To evaluate the computational efficiency of TPA-SSTM, we measured its inference performance using a trained model tested on the Front defense dataset, where it achieves an accuracy of 85.91%. All experiments

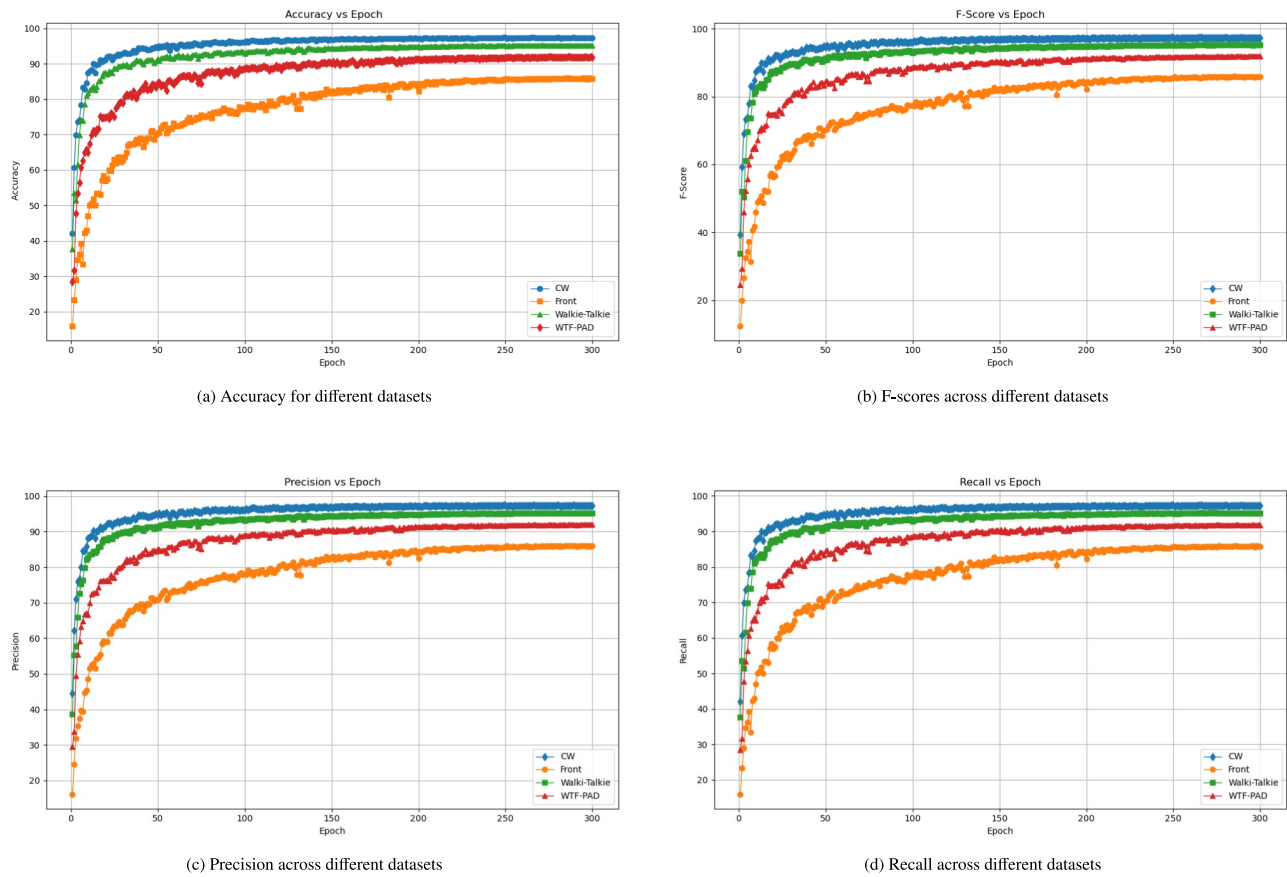


Fig. 5. Evaluation metrics across different datasets.

Component	Specification
CPU	Intel Core i7-10750H @ 2.60 GHz
GPU	NVIDIA GeForce GTX 1660 Ti (6 GB VRAM)
System Memory	32 GB RAM
Framework	PyTorch 2.8.0
CUDA Version	CUDA 12.6

Table 5. Hardware and software configuration used for efficiency evaluation.

were conducted on a consumer-grade laptop, and the input to the model consists of traffic traces with a fixed length of 5000 packets, consistent with the rest of the experimental setup.

The results show that the model requires an average of 4.44 milliseconds to process a single traffic trace, while peak GPU memory usage remains around 100.7 MB. This inference time is shorter than the inter-packet delays typically observed in Tor traffic, which often span several tens of milliseconds. As a result, classification can be performed online without introducing noticeable delays.

Conclusion

This study examines the problem of identifying website access patterns within encrypted tunnels, such as VPN connections, where payload inspection is not feasible. Although website fingerprinting techniques can infer visited websites from traffic metadata, many existing deep learning approaches, particularly those based on convolutional neural networks, show limited robustness against traffic obfuscation defenses. Techniques such as WTF-PAD, Walkie-Talkie, and Front intentionally alter local traffic burst characteristics, which are commonly used as the primary discriminative features in prior models.

To mitigate this limitation, this work introduces Temporal Patch Attention with Self-Supervised Traffic Masking (TPA-SSTM), a hybrid architecture that combines convolutional feature extraction with Transformer-based sequence modeling. The proposed framework employs a semantic patching mechanism that divides encrypted traffic traces into coarse temporal phases, including initial page loading, resource retrieval, and idle or interaction periods. A Transformer encoder is then used to capture long-range dependencies across these

phases, enabling the model to learn global behavioral patterns that remain informative under obfuscation. In addition, self-supervised traffic masking during training, together with data augmentation strategies, improves the model's ability to generalize across defended and non-defended scenarios.

Extensive experiments were conducted on four evaluation settings, including a non-defended Closed World dataset and three defended environments using WTF-PAD, Walkie-Talkie, and Front. The proposed approach achieves 97.4% accuracy in the Closed World setting, 91.9% accuracy under WTF-PAD, and macro F1-scores of 95.25% and 85.91% under Walkie-Talkie and Front, respectively. These findings demonstrate that temporally structured, phase-aware modeling provides substantial gains in robustness when analyzing encrypted traffic protected by obfuscation mechanisms.

The evaluation in this work is performed under a closed-world assumption, where the set of monitored websites is known in advance and remains fixed. This assumption reflects practical censorship and traffic regulation scenarios, in which network operators aim to detect access to a predefined set of restricted domains rather than identify arbitrary or previously unseen websites. Within this context, the proposed framework supports targeted detection of censored content over encrypted communication channels.

Future work will focus on extending the proposed method to open-world settings, where unknown or adversarial websites must be distinguished from known classes. Additional directions include investigating model compression and optimization techniques to support real-time deployment in high-throughput networks, as well as evaluating the robustness of the approach across diverse network environments and geographic regions.

Data availability

The datasets generated during and/or analyzed during the current study will be available upon request.

Appendix A. Training hyperparameters

Table 6 summarizes the hyperparameters used in all experiments for TPA-SSTM.

Parameter	Value
Batch size	128
Optimizer	AdamW
Learning rate (CNN backbone)	1×10^{-4}
Learning rate (Transformer encoder)	1×10^{-4}
Learning rate (Classifier head)	5×10^{-5}
Weight decay	0.05
Label smoothing	0.1
Early stopping patience	40 epochs
Maximum training epochs	300

Table 6. Training Hyperparameters for TPA-SSTM.

Received: 3 November 2025; Accepted: 24 February 2026
Published online: 08 April 2026

References

1. Feng, J., Yu, Y. & Xu, T. Content regulation laws for Chinese isps: Legal responsibilities in free speech and filtering of harmful content. *Law and Economy* **2**(11), 53–59 (2023).
2. Kulesza, J. *Human rights and social media: Challenges and opportunities for human rights education* 139 (Shifting Borders and Liquid Governance, 2024).
3. Hamad, A. & Jia, B. How virtual reality technology has changed our lives: An overview of the current and potential applications and limitations. *Int. J. Environ. Res. Public Health* **19**(18), 11278 (2022).
4. Han, D.-I., Bergs, Y. & Moorhouse, N. Virtual reality consumer experience escapes: Preparing for the metaverse. *Virtual Real.* **26**(4), 1443–1458 (2022).
5. Ametefe, D. S. et al. Advancements and challenges in fingerprint presentation attack detection: A systematic literature review. *Neural Comput. Appl.* **37**(4), 1797–1819 (2025).
6. Xiao, X. et al. A comprehensive analysis of website fingerprinting defenses on Tor. *Comput. Secur.* **136**, 103577 (2024).
7. Razooqi, Y. S. & Pekar, A. Vpn traffic analysis: A survey on detection and application identification. *IEEE Access* **13**(99), 132830–132848 (2025).
8. Bhat, P., Ravi, B. & Saxena, N. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 1923–1940. (2017)
9. Ravensloot, M., Juarez, M. & Goldberg, I. Var-cnn: A data-efficient website fingerprinting attack based on deep learning. *Proceedings on Privacy Enhancing Technologies (PETS)* **4**, 1–20 (2021).
10. Feng, Y. et al. Unmasking the internet: A survey of fine-grained network traffic analysis. *IEEE Commun. Surv. Tutor.* <https://doi.org/10.1109/comst.2025.3545541> (2025).
11. Lyu, Q. et al. Tfan: A task-adaptive feature alignment network for few-shot website fingerprinting attacks on Tor. *Comput. Secur.* **144**, 103980 (2024).
12. Panchenko, A. et al. Website fingerprinting in onion routing based anonymization networks, *ACM IMC*, (2016)

13. Wang, H., Sayadi, H., Sasan, A., Manoj, P. S., Rafatirad, S. & Homayoun, H. Machine learning-assisted website fingerprinting attacks with sidechannel information: A comprehensive analysis and characterization, *2021 22nd International Symposium on Quality Electronic Design (ISQED)*, 79–84. (IEEE, 2021)
14. Juarez, M., Afroz, S., Acar, G., Diaz, C. & Greenstadt, R. A critical evaluation of website fingerprinting attacks. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, Association for Computing Machinery, New York, NY, USA, 263–274 <https://doi.org/10.1145/2660267.2660368>. (2014).
15. Rimmer, V., Preneel, B., Murdock, J., Pesch, D. & Oswald, D. Automated website fingerprinting through deep learning. In: *Network and Distributed System Security Symposium (NDSS)*, The Internet Society, San Diego, CA, USA <https://doi.org/10.14722/ndss.2018.23078>. <https://www.ndss-symposium.org/ndss-paper/automated-website-fingerprinting-through-deep-learning/> (2018).
16. Vaswani, A. et al. Polosukhin, Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*, 5998–6008. (2017)
17. Dosovitskiy, A. et al. An image is worth 16x16 words: Transformers for image recognition at scale, arXiv preprint [arXiv:2010.11929](https://arxiv.org/abs/2010.11929). (2020).
18. Sirinam, P., Imani, M., Juarez, M. & Wright, M. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 1928–1943. (2018)
19. Shin, C.-Y., Choi, Y.-S. & Kim, M.-S. Data augmentation-based enhancement for efficient network traffic classification. *IEEE Access* <https://doi.org/10.1109/access.2024.3525000> (2025).
20. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. & Wojna, Z. Rethinking the inception architecture for computer vision. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2818–2826. (2016)
21. Cui, Y., Jia, M., Lin, T.-Y., Song, Y. & Belongie, S. Class-balanced loss based on effective number of samples. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9268–9277. <https://doi.org/10.1109/CVPR.2019.00950> https://openaccess.thecvf.com/content_CVPR_2019/html/Cui_Class-Balanced_Loss_Based_on_Effective_Number_of_Samples_CVPR_2019_paper.html (IEEE, 2019).
22. Koukoulis, I., Syrigos, I. & Korakis, T. Self-supervised transformer-based contrastive learning for intrusion detection systems, arXiv preprint [arXiv:2505.08816](https://arxiv.org/abs/2505.08816). (2025).
23. Wang, Z. et al. Wf3a: A n-shot website fingerprinting with effective fusion feature attention. *Comput. Secur.* **140**, 103796 (2024).
24. Cai, X. et al. A systematic approach to developing and evaluating website fingerprinting defenses. In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 227–238. <https://doi.org/10.1145/2660267.2660362>. (2014).
25. Wang, T., & Goldberg, I. Walkie-talkie: An efficient defense against passive website fingerprinting attacks. In: *Proceedings of the 25th USENIX Security Symposium*, 137–152. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/wang-tao> (2016).
26. Wang, T. & Goldberg, I. On realistically attacking tor with website fingerprinting. *Proc. Priv. Enh. Technol.* **2016** (4), 21–36 (2016).
27. Lotfollahi, M. et al. Deep packet: A novel approach for encrypted traffic classification using deep learning. In: *Proceedings of the 26th International Conference on Computer Communications and Networks (ICCCN)*, 1–6 <https://doi.org/10.1109/ICCCN.2017.8038460>. (IEEE, 2017).
28. Pan, H., Luo, Q., Wang, Z., Ji, X. & Ren, K. Subverting website fingerprinting defenses with robust traffic representation. In: *Proceedings of the USENIX Security Symposium*, 5585–5602 <https://www.usenix.org/conference/usenixsecurity23/presentation/pa> (2023).
29. Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)* 1–18.
30. Juarez, M., Imani, M., Perry, M., Diaz, C. & Wright, M. Toward an efficient website fingerprinting defense. In: *European Symposium on Research in Computer Security*, 27–46. (Springer, 2016)
31. Cherubin, G., Hayes, J. & Juarez, M. Front: A practical traffic analysis defense for tor. *Proceedings on Privacy Enhancing Technologies (PETS)* **2021**, 1–20. <https://doi.org/10.2478/popets-2021-0028> (2021).
32. Wang, T., & Goldberg, I. {Walkie-Talkie}: An efficient defense against passive website fingerprinting attacks. In: *26th USENIX Security Symposium (USENIX Security 17)*, 1375–1390. (2017)
33. van der Maaten, L. & Hinton, G. Visualizing data using t-sne. *J. Mach. Learn. Res.* **9**, 2579–2605 (2008).
34. Hang, C. et al. Trumorgpt: Graph-based retrieval-augmented large language model for fact-checking. *IEEE Trans. Artif. Intell.* **6**(11), 3148–3162 (2025).
35. Hang, C. et al. Query optimization and semantic reasoning over networks for automated fact-checking In: *58th Annual Conference on Information Sciences and Systems (CISS)*, (2024).

Author contributions

O.A., H.A.; “Conceptualization, validation, methodology, writing—original draft preparation, software”. H.A., W.A. and O.A.; “methodology, review and editing, validation, review and editing”. All authors have read and agreed to the published version of the manuscript.

Funding

No Funding.

Declarations

Competing interest

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to W.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026